	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor:	M.C Rene Adrian Davila Perez
Asignatura:	Programación orientada a objetos
Grupo:	7
No de Práctica(s):	8
Integrante(s):	323036285 323279866 323209432
	323161891 323063609
No. De Lista o brigada	3
Semestre:	2027-1
Fecha de entrega:	23/10/2026
Obervaciones:	

CALIFICACIÓN: _____

Índice

1. Introducción **2**

2. Marco Teórico **3**

3. Desarrollo **5**

4. Resultados **14**

5. Conclusión **16**

1. Introducción

Actualmente los estudiantes de diferentes áreas usan los modelos de lenguaje a gran escala para consultar conceptos técnicos específicos de programación e incluso conceptos aplicados de otras disciplinas, lo que genera un problema ya que muchos estudiantes usan la información sin verificarla y sin saber si es correcta o no, afectando la calidad y confianza del aprendizaje.

Al mismo tiempo, al programar en Java, el uso de interfaces y polimorfismo para modelar clases distintas que comparten un comportamiento común representa un reto para el estudiantado, pues requiere comprender cómo diferentes clases pueden implementar un mismo contrato, definido por la interfaz, de forma independiente y sin duplicar código.

Resolver estos problemas nos ayudará a usar mejor a los LLM a la hora de consultar conceptos técnicos y aplicados, consiguiendo una mejor comprensión y más veracidad en nuestro aprendizaje, también nos ayudará a comprender mejor el uso de interfaces y polimorfismo en el lenguaje de programación Java.

Objetivos

- Elaborar un programa donde las clases Edificio, Auto y Bicicleta implementen la interfaz HuellaCarbono, para aplicar interfaces y polimorfismo correctamente.
- Aplicar la sobreescritura de métodos mediante el uso de @Override, para asegurar que cada clase calcule su huella de carbono y muestre su información con la fórmula y el formato correspondiente.

2. Marco Teórico

Para esta práctica utilizamos el polimorfismo que consiste en conseguir que un objeto de una clase se comporte como cualquiera de sus subclases tanto en métodos como en tipos de datos. Además seguimos viendo lo de herencia que permite a las clases adquirir las propiedades y comportamientos de otras, promoviendo la reutilización del código y el uso de polimorfismo.

`extends`: Palabra reservada en Java para declarar una clase que sea heredada de un caso base.

`instanceof`: Operador que permite verificar si un objeto es instancia de una clase o interfaz determinada.

`toString`: Definido de la clase Object, se utiliza para mostrar la información de un objeto.

`Sobreescritura (@Override)`: redefine el comportamiento de un método específico en una subclase, por lo que tendrá un comportamiento acorde a lo que necesita, necesita tener el mismo nombre de la función a la que sobreescribirá.

`Constructores`: Se pueden usar constructores sobrecargados, pero hay que invocarlo explícitamente.

`super`: permite acceder a los elementos de una clase base.

`Sobrecarga`: Puede definir varios métodos con el mismo nombre, pero con diferentes tipos y números de parámetros.

Además para esta práctica utilizamos el concepto de interfaces.

Una interfaz es una clase abstracta pura, es decir, una clase donde todos los métodos son abstractos (no se implementa ninguno). No se pueden crear instancias de una interfaz. Permiten establecer la forma de una clase (nombres de métodos, listas de argumentos y tipos de retorno), pero no bloques de código [1].

Para utilizarla se utiliza la palabra `interface` en lugar de `class` y puede definirse pública o sin modificador de acceso.

```
public interface NombreInterfaz {
          tipoRetorno nombreMetodo([Parametros]);
          }
```

Todos los métodos que declara una interfaz son públicos y abstractos, también puede contener atributos pero son públicos, estáticos y finales [1].

3. Desarrollo

Ejercicio 1

Para el primer ejercicio se le preguntó a un LLM cómo elaborar una interfaz gráfica de usuario (GUI) básica para inicio de sesión en Java Swing, usando el siguiente prompt:

"Cómo elaborar la GUI para hacer login de usuario y contraseña"

A lo que respondió:

Respuesta de Daniel:

Prompt: "¿Cómo elaborar la GUI para hacer login de usuario y contraseña?".

Para elaborar una GUI de inicio de sesión con usuario y contraseña en Java se utiliza la librería Swing, incluida en el JDK. La interfaz se construye con los siguientes elementos:

- **Ventana principal (JFrame):** Es la ventana de la aplicación. Se configuran su título con `setTitle()`, su tamaño con `setSize()`, su posición centrada con `setLocationRelativeTo(null)` y su cierre con `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`.
- **Panel y administrador de diseño (JPanel y GridLayout):** Dentro de la ventana se coloca un panel que organiza los componentes en filas y columnas, de modo que cada etiqueta quede junto a su campo.
- **Etiquetas (JLabel):** Indican al usuario qué dato debe escribir en cada campo ("Usuario" y "Contraseña").
- **Campo de usuario (JTextField):** Recibe el nombre de usuario en texto visible. Su contenido se obtiene con `getText()`.
- **Campo de contraseña (JPasswordField):** Funciona como un `JTextField`, pero oculta los caracteres con puntos. Su contenido se obtiene con `getPassword()`, que devuelve un arreglo de `char`; se convierte a `String` con `new String(...)` para poder compararlo.

- **Botón (JButton):** Es el elemento que el usuario presiona para iniciar sesión.
- **Manejo del evento (ActionListener):** Se asocia al botón con `addActionListener()`. Al hacer clic se leen ambos campos y se comparan con las credenciales registradas, usando `equals()` y no `==`.
- **Mensajes al usuario (JOptionPane):** Si las credenciales son correctas, se muestra un mensaje de bienvenida, se cierra la ventana de login con `dispose()` y se abre la ventana principal del sistema. Si son incorrectas, se muestra un mensaje de error y se limpia el campo de contraseña con `setText()` para que el usuario intente de nuevo.
- **Mostrar la ventana:** Se invoca `setVisible(true)` dentro de `SwingUtilities.invokeLater` para que la interfaz se ejecute en el hilo de eventos de Swing.

Como buena práctica, las credenciales no deben escribirse directamente en el código ni guardarse en texto plano: en un sistema real se validan contra un archivo o una base de datos, almacenando la contraseña cifrada (*hash*).

Respuesta de Germán:

Para elaborar una Interfaz Gráfica de Usuario (GUI) orientada a un sistema de inicio de sesión con usuario y contraseña utilizando Java Swing, debes estructurar los elementos visuales y la lógica de validación de la siguiente manera:

- **Contenedor Principal (JFrame):** Funciona como la ventana principal de la aplicación donde se agrupan todos los componentes visuales de acceso.
- **Etiquetas descriptivas (JLabel):** Se colocan en la interfaz para indicar claramente al usuario dónde debe ingresar su nombre de usuario y su contraseña.
- **Campo de texto (JTextField):** Permite al usuario escribir y visualizar su nombre de usuario o identificador en formato de texto plano.
- **Campo de contraseña (JPasswordField):** Oculta automáticamente los caracteres ingresados mediante asteriscos o puntos para garantizar la seguridad de la clave.

- **Botón de acción (JButton):** Activa el evento de inicio de sesión al hacer clic sobre él.
- **Validación de credenciales:** Al presionar el botón, el sistema captura los datos introducidos en ambos campos y los compara con los registros o la lógica de control de usuarios.
- **Control de flujo y navegación:** Si los datos son correctos, se muestra un mensaje de éxito, se cierra la ventana de acceso y se abre la interfaz principal del sistema; de lo contrario, se despliega una alerta de error indicando que las credenciales son incorrectas.

Respuesta de Giovani:

"Toma los conocimientos básicos de polimorfismo, clases y herencia en Java, cómo los podrías utilizar para elaborar una GUI básica donde uno pueda ingresar un usuario y contraseña, tanto para registrar a un nuevo usuario como para iniciar sesión, habrá que utilizar Java Swing para ello y necesitará una interfaz gráfica".

- Los usuarios viven en memoria. Al cerrar el programa se pierden. `RepositorioUsuarios` está aislado para que luego puedas guardarlos en un archivo o base de datos sin tocar la GUI.
- El primer usuario registrado es administrador y los demás son estándar. Es solo para que el polimorfismo se vea en acción.
- Contraseñas: nunca se guardan en texto plano, sino con SHA-256 y sal aleatoria. Para un proyecto real se recomienda un algoritmo diseñado para esto, como PBKDF2, bcrypt o Argon2, porque SHA-256 es demasiado rápido y vulnerable a fuerza bruta.
- Validaciones: campos vacíos, usuario duplicado, contraseña de al menos 6 caracteres y mensaje genérico en login fallido, para no revelar si el usuario existe.

Respuesta de Javier:

Para elaborar una Interfaz Gráfica de Usuario (GUI) de inicio de sesión en Java, se emplea la librería estándar Swing, ideal para aplicaciones de escritorio. Los pasos clave son:

- **Contenedor principal (JFrame):** Define la ventana de la aplicación. Se configuran el título, las dimensiones mediante `setSize()` y la acción de cierre con `setDefaultCloseOperation()`.
- **Panel contenedor (JPanel):** Se ubica dentro del marco para organizar los elementos visuales usando administradores de diseño como `GridLayout` o `FlowLayout`.
- **Campos de entrada (JTextField y JPasswordField):** Se usan junto con etiquetas (`JLabel`) para capturar el usuario y la contraseña. El campo de clave enmascara los caracteres automáticamente por seguridad.
- **Botón de acción (JButton):** Elemento interactivo que el usuario presiona tras rellenar sus credenciales.
- **Validación de eventos (ActionListener):** Se asocia al botón para capturar los datos ingresados, compararlos con los valores almacenados y permitir el acceso o mostrar un mensaje de error.

Finalmente, se invoca el método `setVisible(true)` sobre el marco para mostrar la ventana en pantalla, completando una interfaz funcional y segura.

Respuesta de Leslie:

Usaré **Java con Swing**, la librería de ventanas que viene incluida en Java. Reutiliza la clase Usuario de la guía anterior.

Pasos

1. Crea una clase `LoginGUI` que herede de `JFrame` (la ventana).
2. Agrega los componentes: etiquetas (`JLabel`), caja de usuario (`JTextField`), caja de contraseña (`JPasswordField`) y botón (`JButton`).
3. Al presionar el botón, toma lo escrito y usa validar de `Usuario`.
4. Muestra el resultado en una etiqueta.

Qué hace cada parte

- **JFrame:** la ventana. Al heredar de ella, tu clase es una ventana.
- **JPasswordField:** como `JTextField`, pero oculta lo que se escribe con puntos.
- **addActionListener:** le dice al botón qué hacer cuando lo presionan.
- **GridLayout:** acomoda los componentes en filas, uno debajo del otro.

Importante: guarda `Usuario.java` y `LoginGUI.java` en la misma carpeta (o paquete) para que se vean entre sí.

Ejercicio 2

Para empezar el programa, primero se le preguntó a un LLM: "¿Cómo calculo la huella de carbono de un edificio, auto y bicicleta?", tras analizar la respuesta, a cada miembro del equipo la que más nos satisfizo fue la siguiente:

El cálculo de la huella de carbono se fundamenta en la fórmula estándar del Panel Intergubernamental sobre el Cambio Climático (IPCC):

```
HuellaDeCarbono(kgCO2e)=DatoDeActividad*FactorDeEmision
```

1. Edificio

Las emisiones de un edificio provienen principalmente del consumo de energía estacionaria: electricidad de la red y gas para calefacción o agua caliente.

Factores de emisión estándar:

- Electricidad (Factor oficial CRE/SEMARNAT en México): 0.438 kg-CO2e/kWh.

Fórmula implementada:

```
Huella = consumoElectricoKwh * 0.438
```

2. Auto

La huella de un automóvil de combustión interna depende del combustible quemado al desplazarse.

Factores de emisión estándar:

- Gasolina: 2.31kgCO2e/litro.

Fórmula:

```
LitrosTotales=kmRecorridosAnuales/rendimientoKmPorLitro
```

```
Huella=LitrosTotales*2.31
```

3. Bicicleta

Una bicicleta mecánica no quema combustible ni consume energía eléctrica en su operación diaria (su emisión directa por tubo de escape es 0.0). Sin embargo, para fines de modelado en software existen dos alternativas realistas:

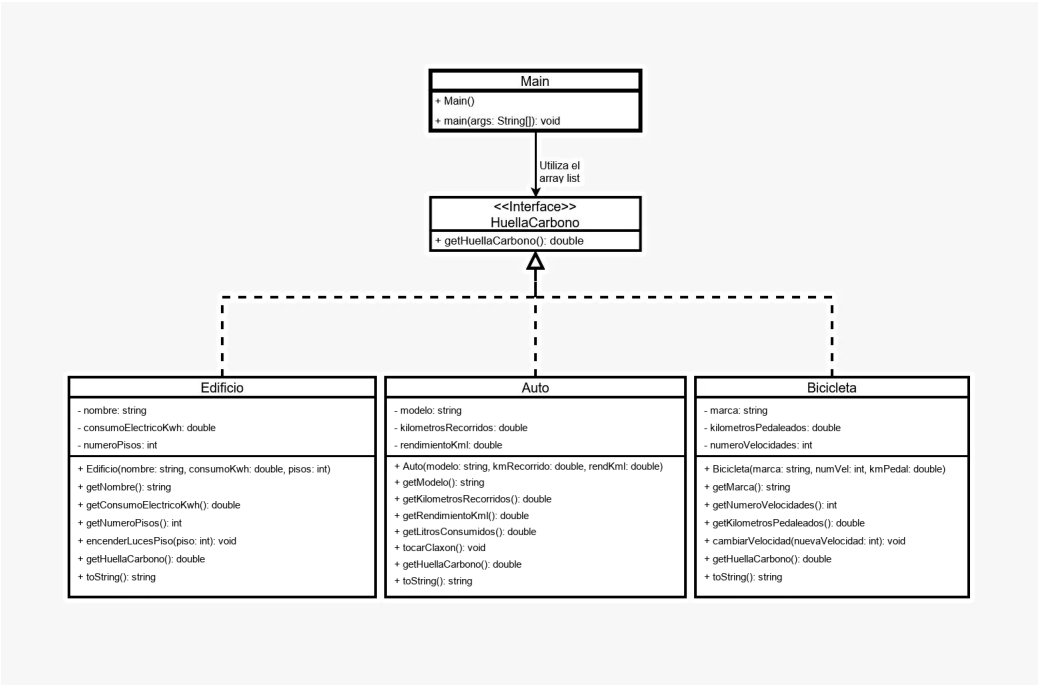
- **Opción A (Bicicleta convencional - Ciclo de vida / Manufactura amortizada):** La Federación Europea de Ciclistas (ECF) estima que la huella de fabricación y mantenimiento de una bicicleta tradicional equivale a unos 0.005kgCO2e por km recorrido.

  ```
  Huella=kmRecorridos*0.005
  ```

- **Opción B (Bicicleta eléctrica):** Se calcula a partir de los kWh consumidos al recargar la batería en la toma de corriente.

  ```
  Huella=kwhBateriaAnual*0.50
  ```

Para la realización del programa se siguió el siguiente UML:



Para iniciar se creó una interfaz `HuellaCarbono`, donde se estableció que cada clase que implemente esta interfaz debe incluir un método llamado `getHuellaCarbono`, que retorna un dato de tipo `double`.

Después de esto, se creó una clase `Bicicleta`, la cual implementa la interfaz `HuellaCarbono`. Esta clase tiene como atributos `marca` de tipo `String`, `numeroVelocidades` de tipo `int` y `kilometrosPedaleados` de tipo `double`, todos encapsulados de forma privada. Posteriormente se estableció el constructor de la clase, donde se le asigna un dato a cada atributo. Tras programar el constructor, se procedió a programar los getters de cada atributo, los cuales retornan lo que está guardado en cada atributo del objeto. Después se programó el método `cambiarVelocidad`, que no retorna ningún valor; este método recibe como parámetro un número entero que representa la velocidad a la que el usuario desea cambiar, y con un ciclo `if-else` se valida que la velocidad esté en el modelo de bicicleta. Posteriormente se usó un `@Override`, sobreescribiendo el método `getHuellaCarbono` de la interfaz; el valor de tipo `double` que retorna es la cantidad de kilómetros pedaleados multiplicado por 0.005. Finalmente se usó nuevamente `@Override` para sobreescribir `toString` de la clase `Object`; retorna una cadena de texto con la información de la bi-

cicleta.

Posteriormente, se creó una clase `Auto`, que implementa la interfaz `HuellaCarbono`. Esta clase tiene como atributos `modelo` de tipo `String`, `kilometrosRecorridos` de tipo `double` y `rendimientoKml` de tipo `double`, todos encapsulados de forma privada. Después de establecer los atributos, se estableció el constructor de la clase, donde se le asigna un dato a cada atributo. Después de la programación del constructor, se continuó programando los getters de cada atributo, los cuales retornan lo que está guardado en cada atributo del objeto. Se procedió a programar el método `getLitrosConsumidos`, que retorna un valor de tipo `double`; con una estructura `if-else` se verifica que `rendimientoKml` sea mayor a 0. Si esto se cumple, se retorna la división de `kilometrosRecorridos` entre `rendimientoKml`, pero si no es así se retorna 0.0. Posteriormente se usó un `@Override`, sobreescribiendo el método `getHuellaCarbono` de la interfaz; el valor de tipo `double` que retorna es la cantidad de litros consumidos multiplicado por 2.31. Finalmente se usó nuevamente `@Override` para sobreescribir `toString` de la clase `Object`; retorna una cadena de texto con la información del auto.

Después, se creó una clase `Edificio`, que también implementa la interfaz `HuellaCarbono`. Esta clase tiene como atributos `nombre` de tipo `String`, `consumoElectricoKwh` de tipo `double` y `numeroPisos` de tipo `int`, todos encapsulados de forma privada. Después de establecer los atributos, se estableció el constructor de la clase, donde se le asigna un dato a cada atributo. Posterior a la programación del constructor, se continuó programando los getters de cada atributo, los cuales retornan lo que está guardado en cada atributo del objeto. Se procedió a programar un método `encenderLucesPiso`, que no retorna ningún valor y recibe como parámetro un número de piso; con una estructura `if-else` se verifica que el piso exista en el edificio. Si esto se cumple, se imprime un mensaje diciendo que las luces están prendidas en ese piso, pero si no es así se imprime un texto mencionando que el piso no existe en el edificio. Posteriormente se usó un `@Override`, sobreescribiendo el método `getHuellaCarbono` de la interfaz; el valor de tipo `double` que retorna es el consumo eléctrico en kWh multiplicado por 0.438. Finalmente se usó nuevamente `@Override` para sobreescribir `toString` de la clase `Object`; retorna una cadena de texto con la información del edificio.

Finalmente, en la clase `Main` se comenzó programando el instanciamiento

de un objeto de cada clase, invocando el constructor y dándole un valor a los atributos. Después se invocaron los métodos propios de cada clase. Posteriormente se programó un `ArrayList` con los objetos que contienen el método huella de carbono, para finalmente imprimir la huella de carbono de cada uno.

4. Resultados

Para comprobar el funcionamiento correcto, en la consola podemos observar la ejecución de las clases.

Edificio: Su comportamiento exclusivo imprimirá cuántas luces están encendidas en el piso que se le especificó; en caso de no darle un número válido, indicará que ese piso no existe.

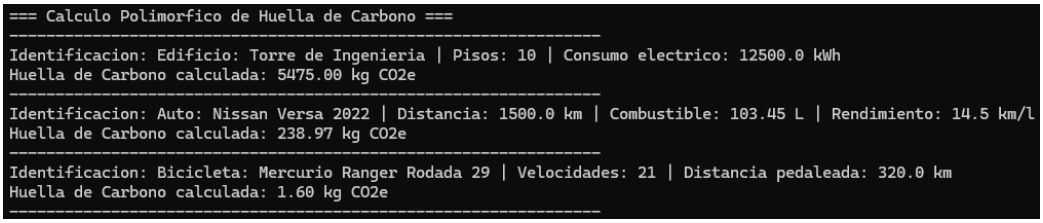
Auto: Como comportamiento exclusivo tendrá un claxon, el cual hará sonar cada vez que se le indique.

Bicicleta: En este caso asumimos que tiene una palanca de cambios para variar su velocidad, por lo tanto sus comportamientos exclusivos serán establecer el número de velocidades que tiene y los cambios de velocidad que haya en ella. El mismo principio de las luces aplica; en caso de mandar un número fuera de rango, indicará que no es posible.

```
=== Demostracion de Comportamientos Exclusivos ===
Luces encendidas en el piso 4 del edificio Torre de Ingenieria
El auto Nissan Versa 2022 suena: Beep beep!
Se cambio a la velocidad 7 en la bicicleta Mercurio Ranger Rodada 29
```

Para el cálculo de las emisiones de carbono se sobreescribirá el método `getHuellaCarbono()` definido en la interfaz, implementando la fórmula correspondiente en cada clase:

- **Bicicleta:** `kilometrosPedaleados * 0.005` (Factor indirecto de manufactura/desgaste).
- **Edificio:** `consumoElectricoKwh * 0.438` (Factor oficial del Sistema Eléctrico Nacional).
- **Auto:** `(kilometrosRecorridos / rendimientoKml) * 2.31` (Litros consumidos multiplicados por emisión de gasolina).



Los objetos creados con base en las clases mencionadas se almacenan dentro de una lista polimórfica (`ArrayList<HuellaCarbono>`), recorriéndola para mostrar tanto su identificación como la huella de carbono que produce cada entidad.

5. Conclusión

Se cumplieron los objetivos de la práctica al implementar la interfaz HuellaCarbono en las clases Edificio, Auto y Bicicleta. El uso del polimorfismo y @Override demostró cómo cada objeto puede calcular sus propias emisiones de forma independiente, evitando la duplicación de código. Finalmente, se usaron correctamente los Modelos de Lenguaje para validar las fórmulas ambientales y ayudarnos a resolver el ejercicio de manera satisfactoria.

Referencias

[1] M. A. Nakayama et al. *Manual de prácticas del laboratorio de programación orientada a objetos*. Facultad de Ingeniería, Universidad Nacional Autónoma de México. 2017. Código MADO-22, versión 01.